

Quantum Circuit Synthesis from C via Multi-Level Intermediate Representation

Giacomo Lancellotti

Politecnico di Milano

Milano, Italy

giacomo.lancellotti@polimi.it

Leonardo Ignazio Pagliochini

Politecnico di Milano

Milano, Italy

leonardoignazio.pagliochini@mail.polimi.it

Francesco Rosnati

Politecnico di Milano

Milano, Italy

francesco.rosnati@mail.polimi.it

Giovanni Agosta

Politecnico di Milano

Milano, Italy

giovanni.agosta@polimi.it

Abstract

Programming quantum computers is a complex endeavor that requires highly specialized knowledge. Given the growing momentum of quantum computing, high-level synthesis of quantum circuits from well-known classical languages could significantly benefit the ecosystem, allowing specialists to focus on the parts of an application where quantum computing offers the greatest advantage, while letting the compiler handle the boilerplate code. To bridge this gap, we present a high-level synthesis framework that translates C programs into quantum circuits through a sequence of modular and structured Multi-Level Intermediate Representation (MLIR) dialects. These dialects expose classical control and data flow in a form compatible with quantum constraints, enabling reversible and unitary circuit generation. Our framework maps conditionals, loops, and arithmetic operations, supports configurable bitwidths, and allows retargetability to different quantum backends. Furthermore, we demonstrate a correlation between classical program metrics—such as instruction count and cyclomatic complexity—and the performance characteristics of the resulting quantum circuits. To validate our methodology, we quantify compilation resource usage, showing a linear scaling of memory footprint and a quadratic scaling of CPU time as a function of quantum circuit complexity. Our tool generates circuits in OpenQASM format, facilitating integration with current quantum programming frameworks.

CCS Concepts

• **Software and its engineering** → **Source code generation**; • **Hardware** → **Quantum computation**.

Keywords

High-level synthesis, quantum circuit generation, C programming, compiler design, MLIR

ACM Reference Format:

Giacomo Lancellotti, Francesco Rosnati, Leonardo Ignazio Pagliochini, and Giovanni Agosta. 2026. Quantum Circuit Synthesis from C via Multi-Level Intermediate Representation. In *The 41st ACM/SIGAPP Symposium on Applied Computing (SAC '26)*, March 23–27, 2026, Thessaloniki, Greece. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3748522.3779947>

1 Introduction

Quantum systems are moving beyond prototype, research-only devices toward fault-tolerant platforms that can run useful applications with a computational advantage over the best classical solutions [31, 1]. Achieving this requires a significant increase in the number of physical qubits, the efficient implementation of error-correction protocols, and the seamless integration of quantum hardware with classical infrastructures like high-performance computing systems [24]. Consequently, quantum software engineering has become an active area of research, encompassing quantum-specific programming languages, compilers, and orchestration platforms for quantum-classical integration [9, 17, 23].

Since quantum programming combines quantum-specific operations with classical control structures—such as arithmetic, decision branches, and loops—the latter are usually executed externally on a classical controller, relying on mid-circuit measurements for conditional operations. This is typically due to the lack of integration of these constructs within the quantum computing environment, which often increases the cost of back-and-forth communication with the classical host and results in the loss of opportunities for global quantum program optimization.

Related Work. To address the lack of a standard IR for quantum computation and enable scalable optimization, [22] proposed one of the first integrations of quantum programming languages into the Multi-Level Intermediate Representation (MLIR) compilation toolchain [18], designing dialects for OpenQASM [6] and Quil [32]. Similarly, [15, 3] define detailed SSA-based IRs with custom MLIR dialects and multi-phase lowering. However, both of these efforts target quantum-native representations, such as the Q# [33] language or Qiskit [29] source code. By contrast, the entry point of our work is standard C, which pushes the abstraction boundary upward and enables the synthesis of quantum subroutines directly from classical logic. Additionally, [20] proposed QHLS, a C-to-quantum



This work is licensed under a Creative Commons Attribution 4.0 International License. SAC '26, Thessaloniki, Greece

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2294-3/2026/03

<https://doi.org/10.1145/3748522.3779947>

synthesis framework. QHLS relies on ad hoc transformations instead of MLIR, which limits its modularity and optimization potential. In contrast, our pipeline supports flexible dialect design, exposes intermediate layers, and enables modular optimizations.

This Work. We present a MLIR-based compilation toolchain that enables a tighter integration between classical and quantum programming. Inspired by principles from High-Level Synthesis (HLS), our toolchain directly synthesizes quantum circuits from classical C programs. The synthesis process leverages a sequence of quantum-aware Intermediate Representations (IRs), expressed as MLIR dialects, which preserve control and data flow while respecting quantum constraints such as reversibility and the no-cloning theorem. Unlike hybrid models, where classical processors merely orchestrate isolated quantum subroutines, our approach enables the construction of complex, self-contained quantum routines.

This work is motivated by the question of whether classical programming abstractions can be systematically and reliably lifted into the quantum domain. Specifically, we investigate whether a compiler grounded in MLIR can provide a principled and general method for translating conventional C constructs into quantum-compatible representations, without relying on complex transformations or making domain-specific assumptions. Additionally, we explore whether this framework can reveal meaningful relationships between the structure of classical programs and the resource requirements of the resulting quantum circuits. These research questions guide both the design of our toolchain and the analyses presented in this paper.

Contributions. We introduce a modular MLIR-based compilation infrastructure that synthesizes quantum circuits directly from high-level C programs, emitting the circuits in OpenQASM format for interoperability with current quantum frameworks. We define two intermediate representations: a *Quantum-Aware IR* and a *fully Quantum IR*, which preserve high-level abstractions throughout the compilation process, enabling optimizations beyond single-qubit gate transformations. We provide, to the best of our knowledge, the first analysis in the literature examining the relationship between classical program metrics and quantum resource usage. In particular, we show that quantum resource metrics—qubit count, gate count, and circuit depth—scale linearly with the program instruction count, while growing exponentially with cyclomatic complexity when mid-circuit measurements are not employed.

To assess the feasibility of our toolchain, we evaluate its classical resource requirements in terms of CPU time and memory consumption. Our analysis shows that CPU time scales quadratically, while memory usage grows linearly with respect to circuit depth and a composite structural metric. The full implementation and benchmark suite are available at <https://github.com/leonardopagliochini/QuantumC>.

Structure of the paper. The remainder of this work is organized as follows. Section 2, *Background*, reviews the fundamentals of quantum computing, high-level synthesis, and the MLIR framework. Section 3, *MLIR Compilation Toolchain*, presents the proposed compilation pipeline, detailing the intermediate representations and the translation of C primitives. Section 4, *Results and Discussion*, reports the experimental evaluation, covering both quantum resource usage and classical compilation requirements. Section 5, *Conclusions*, concludes the paper.

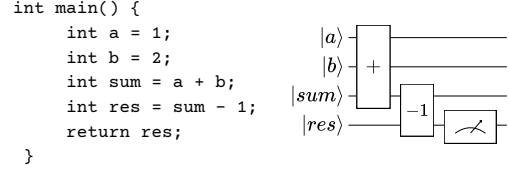


Figure 1: Example of a simple C program and its corresponding quantum circuit. Each variable is mapped to a quantum register, while the operations are abstracted as quantum gates representing addition and constant subtraction. Return value is obtained through a measurement operation.

2 Background

In this section, we introduce the basic concepts of quantum computation and discuss how classical HLS principles can be extended to the generation of quantum circuits. Finally, we present the MLIR-based compiler framework on which our methodology is built.

2.1 Quantum Computing Basics

In quantum computing, a *qubit* represents the fundamental unit of information, whose state is defined as a two-dimensional vector in the Hilbert space $\mathcal{H} = \mathbb{C}^2$ [27]. Using the bra-ket notation, it represents a vector given by the linear combination—i.e. the *superposition*—of two basis states $|0\rangle$ and $|1\rangle$, resulting in the state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex probability amplitudes such that $|\alpha|^2 + |\beta|^2 = 1$. For a multiple-qubit system, the state is defined as $|\psi\rangle_n = \sum_{i=1}^{2^n} \alpha_i |i\rangle$ in the Hilbert space $\mathbb{C}^{2^n}$. To extract information from a quantum system, we apply a measurement operator that collapses the system state into one of the 2^n basis states with a probability equal to the square of the corresponding amplitude.

To evolve the state of a system, we apply *quantum operators*, which are unitary matrices in $\mathbb{C}^{2^n \times 2^n}$. The evolution of a quantum state is given by $|\psi_{i+1}\rangle = U|\psi_i\rangle$, representing the standard matrix-vector multiplication between the operator U and the state $|\psi_i\rangle$. Since quantum operators are unitary, applying them in reverse (i.e., using $U^\dagger$) allows one to recover the original state. This fundamental property implies that quantum computation is inherently *reversible*.

In this work, we adopt the *circuit formalism* to represent quantum computation. In this model, qubits—often referred to as *virtual qubits*—are depicted as horizontal lines that can, for clarity, be grouped into *quantum registers* of arbitrary size. Quantum operations are expressed as *gates* applied to one or more qubits, and the computation proceeds from left to right. Computation is achieved by applying a sequence of quantum gates that modify the system state to increase the probability of measuring the quantum state associated with the correct outcome of the algorithm. Typically, quantum computation is expressed using a universal quantum gate set, such as *Clifford+T*, which allows one to approximate any quantum gate within a specified error threshold [27], much like a complete set of classical logic gates enables universal classical computation.

It is particularly useful in quantum algorithm design to abstract the functionality of a quantum gate, which may represent a single-qubit operation or an entire subroutine acting at the register level.

This abstraction allows complex algorithms to be expressed in a more compact and intuitive form. An example of such a representation is shown in Fig. 1.

2.2 High-Level Synthesis for Quantum Circuits

HLS is the process of automatically translating high-level programming descriptions, such as C code, into low-level hardware designs which is typically expressed as Register-Transfer Level (RTL) representations [4]. The main goal of HLS is to accelerate hardware design by enabling specialists to focus on the functional behavior of the system while delegating physical constraints and optimizations—such as area, timing, and power consumption—to automated synthesis tools [11]. This approach has found widespread application in domain-specific designs, including hardware accelerators for machine learning and cryptography [13, 8].

Similarly, a quantum circuit can be synthesized starting from a high-level description. The synthesis translates high-level operations into their quantum counterparts to optimize the performance of the resulting circuit. In this context, several metrics can be adopted to evaluate the quality of a design: the total number of virtual qubits required and the number of quantum gates along the circuit critical path, commonly referred to as the *circuit depth*.

In Fig. 1, we present an example of C-to-quantum synthesis. When performing quantum synthesis, additional constraints unique to quantum computation must be considered. For instance, the no-cloning theorem forbids the duplication of arbitrary quantum states [34], thereby eliminating the classical notion of fan-out for variable duplication. Moreover, since quantum computation is reversible, information cannot be unitarily erased [28]. Consequently, a register holding a variable cannot be overwritten with a new value unless the previous value has been explicitly uncomputed. This often requires the use of ancillary data structures to store intermediate results during the uncomputation process. Classical constructs such as arithmetic units, loops, and conditional branches, if not carefully handled, can rapidly inflate quantum resource usage. For this reason, synthesis tools typically rely on multiple IRs defined at different levels of abstraction—from single-qubit operations to entire functions—to enable efficient classical-to-quantum mapping strategies.

2.3 Multi-Level Intermediate Representation

To address the challenges posed by heterogeneous hardware and domain-specific languages—such as those used in quantum computing—MLIR [18] was introduced to simplify compiler development by providing a common, extensible interface. MLIR is a flexible framework designed to represent and optimize programs across multiple levels of abstraction, called *dialects*, each with its own namespace, types, and semantic rules. It is built on the principle of *progressive lowering*, where high-level dialects are gradually transformed into lower-level ones through a series of transformations that are legal with respect to the target representation. This approach provides a high degree of modularity, allowing optimizations to be implemented at the appropriate level of abstraction without interference from other representations.

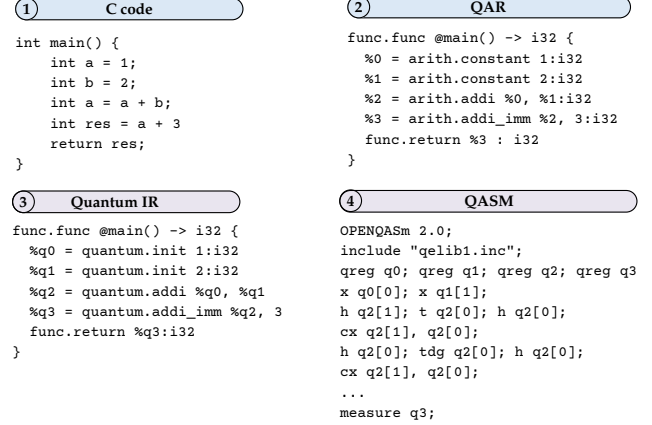


Figure 2: Progressive lowering of a simple C program through the proposed compilation pipeline. Starting from the C source (step 1), the program is first translated and regularized into QAR (step 2), then lowered to the QR (step 3), where quantum semantics are introduced, and finally emitted as an OpenQASM file (step 4).

3 MLIR Compilation Toolchain

In the following section, we introduce our compilation pipeline, explain the design choices behind our IRs and provide an overview of how classical C primitives are synthesized into quantum circuit components.

3.1 Abstractions and Intermediate Representations

In Fig. 2 we report our compilation pipeline progressing from C programs to backend-executable quantum circuits.

AST Generation. The first step involves parsing the C source code and generating an abstract syntax tree (AST) that represents both the syntactic structure and semantic content, including variable declarations, arithmetic expressions, and control-flow constructs. The raw AST is then normalized into a structured dataclass tree, filtering out irrelevant metadata to simplify traversal during IR generation.

Quantum-Aware Representation (QAR). The QAR is constructed directly from the dataclass tree and serves as a regularization step for subsequent lowering. It does not encode quantum operations; rather, it ensures that classical constructs—such as variable assignments, conditional statements, and loops—are expressed in a quantum-safe manner, preventing any violations of quantum constraints. For instance, QAR is based on the Static Single Assignment (SSA) form. Beyond being a common best practice in compiler design, SSA prevents variable reuse, which guarantees that all operations are modeled as out-of-place; this prevents variable overwriting and preserves intermediate values, thereby maintaining reversibility at every step of the computation. An example of this is shown in Fig. 2.

Quantum Representation (QR). The QAR is lowered into the QR, which explicitly encodes quantum semantics. This dialect operates at a high level of abstraction, working directly on quantum registers

Table 1: Supported C Primitives

Expr.	Literals / References	Integer literals, scalar vars
	Unary Ops	not, plus, dec
	Binary Arith.	add, sub, mul, div
	Comparison	eq, neq, etc.
Control	Logical Conn.	and, or
	Branching	if-else
	Loops	for, while

rather than on individual qubits. This design preserves flexibility regarding low-level implementation details, such as the bitwidth encoding of variables within a register. Moreover, QR is independent of specific quantum backends. For example, in arithmetic routines, the choice between a Quantum Fourier-based implementation [30] and a ripple-carry implementation [7] can be deferred to a later stage, allowing the most efficient approach to be selected for the target backend. An example of QR is illustrated in Fig. 2.

Quantum Backend Lowering. The QR is ultimately compiled into the target quantum circuit. High-level operation abstractions are translated into quantum gates according to a specific gate set, such as Clifford+T. Arithmetic implementations are selected to best match the hardware requirements, ensuring efficient execution. The resulting circuit is output in OpenQASM [6] format to guarantee compatibility and interoperability with existing quantum programming frameworks.

3.2 C-Primitives Translation

The list of supported expression and control flow statement is reported in Tab. 1.

Data Representation. During synthesis, we preserve the C-type notation so that each variable is allocated as a quantum register of chosen width (e.g., 16 or 32 qubits) rather than being mapped directly onto individual qubits. This register-level abstraction mirrors the classical representation, enabling optimizations at a higher semantic level and allowing flexible management of operation precision [14, 12], which can be particularly useful in quantum machine learning and approximate computing applications [5].

Variable Assignment and Declaration. In classical C programs, variables can be freely overwritten, since the operating system handles register allocation and memory updates. In contrast, quantum programs must preserve reversibility. For example, in Fig. 2, the variable *a* is reused to store the result of an addition after its declaration. Since QAR is designed in SSA form, the result of the addition is stored in a newly allocated variable, *%2*. Variables are therefore never overwritten, inherently preserving reversibility and allowing them to be later uncomputed. The same principle is maintained through successive lowering stages to QR and final circuit generation, where a new quantum register, *q2*, is allocated to hold the result of the addition.

Arithmetic Operations. Arithmetic operations such as add, sub, mul, and div are represented as opaque definitions in both QAR and QR using constructs like *arith.op*. The actual implementation in the quantum circuit is deferred until the final synthesis step, when QR is lowered to QASM. This strategy allows earlier

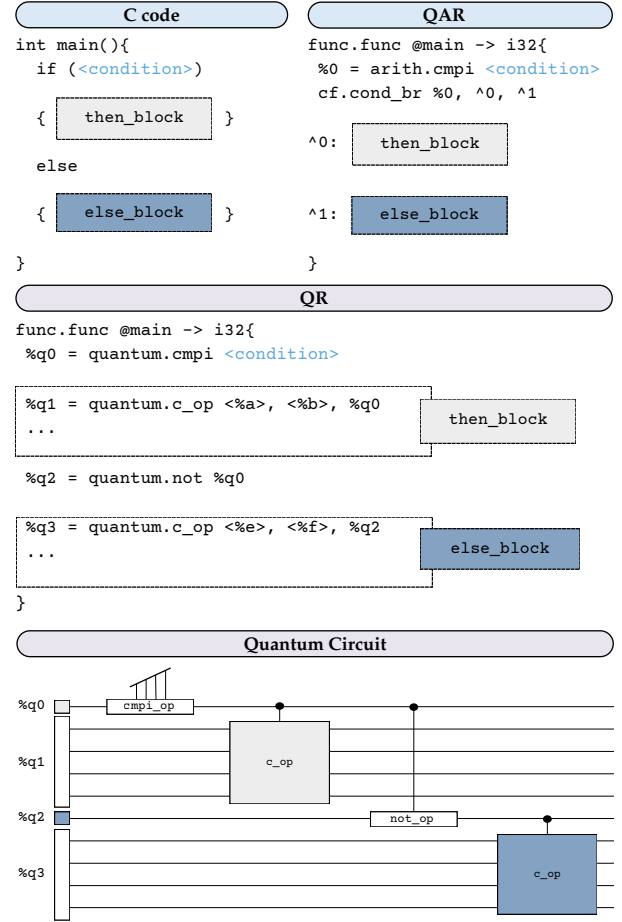


Figure 3: Translation of an if-then-else construct across compilation stages, showing how classical control flow is progressively mapped to quantum-controlled subcircuits.

stages of the pipeline to focus on structural translation without committing to a specific arithmetic implementation. Different implementations—such as ripple-carry logic [7] or QFT-based methods [30, 16]—present distinct trade-offs in circuit depth, ancilla usage, and precision. Because arithmetic operations are often used repeatedly, these choices can have a significant impact on overall performance. Deferring the implementation enables users to select the most suitable design for the target quantum backend.

In current implementation, we do not explicitly model arithmetic overflow within quantum registers, as the quantum backend used for validation in our experiments detects overflow conditions internally and aborts execution when they occur. Consequently, all benchmark programs were chosen or parameterized to remain within representable ranges, ensuring that no overflow arises during synthesis or simulation. A systematic treatment of overflow semantics—such as defining explicit modular or bounded behavior—remains outside the scope of this work and represents a direction for future extensions of the framework.

Immediate Operations. When one operand is known at compile time, the compiler detects it during AST traversal and emits an immediate operation, `op_imm`, such as `add_imm` shown in Fig. 2. In QAR, the constant operand is encoded as a literal parameter rather than as a separate quantum register, following the classical concept of immediate instructions in processor architectures. Introducing immediate operations serves two main purposes. First, it reduces register pressure by avoiding the allocation of dedicated quantum registers for constants. Second, it enforces a clear semantic distinction between variable-variable and variable-constant operations. The resulting circuits preserve the same semantics while remaining structurally leaner.

Conditional Branching. Conditional constructs in QAR are represented as explicit SSA conditionals, `cf.cond_br`, using dedicated if-else nodes; an example is shown in Fig. 3. When the condition is computed, e.g., `%0 = arith.cmpi <condition>`, the resulting value `%0` serves as the guard of the conditional branch. Depending on the boolean outcome, control flow jumps to either the then or else block. Compound conditions (e.g., `&&` or `||`) are normalized before lowering: conjunctions are represented as nested conditionals, while disjunctions are expressed as duplicated branches that converge to the same body, ensuring that control flow remains fully explicit.

In QR, conditional branching is implemented using controlled quantum operations. The boolean outcome of a comparison is promoted to a dedicated control qubit, and the corresponding branch body is executed only when this qubit evaluates to true. For example, the operation `%q0 = quantum.cmpi <condition>` determines whether operations on `%q1` or `%q3` are executed, depending on the states of the control qubits `%q0` and its negation `%q2`. Cascaded or nested conditionals are represented by chaining such controlled operations. The final circuit, illustrated in Fig. 3, shows how all then and else blocks are translated into controlled quantum gates. This approach ensures that the entire circuit is synthesized using only quantum operations, thereby avoiding mid-circuit measurements as in [20], which would otherwise require interaction with the classical controller.

Loops. Similar to conditional branches, loops are unrolled into a sequence of comparison operations, `arith.cmpi`; an example is shown in Fig. 4. If the loop bound is known only at runtime, a maximum iteration limit (`max_iter`) is set. The loop body is then emitted up to this maximum number of iterations, which allows us to avoid mid-circuit measurements for runtime condition evaluation. During the conversion from QAR to QR, the same strategy used for if statements is applied: each operation inside the unrolled loop body is transformed into a controlled operation, conditioned on the loop predicate evaluated at the corresponding iteration.

4 Results and Discussion

In this section, we summarize the tools employed, describe our testing strategy, and provide an analysis and discussion of the resulting performance.

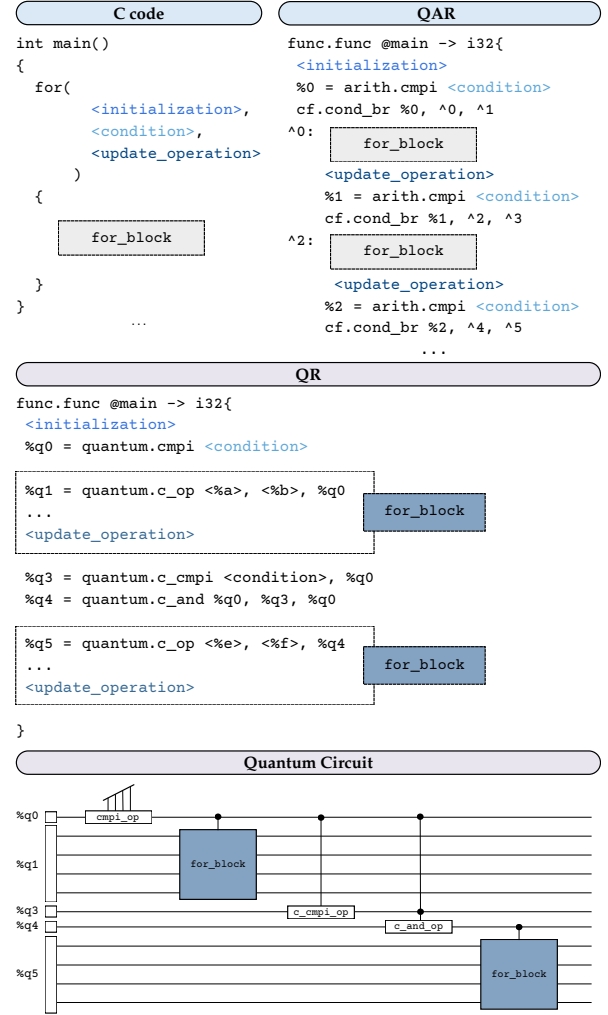


Figure 4: Lowering of a for-loop across the compilation flow, from QAR to QR and finally to the resulting quantum circuit.

4.1 Experimental Setup

We parse the C source programs using a Clang-based frontend [19] to generate the corresponding AST. For MLIR generation, we employ xDSL (version 0.54.0) [10], a Python-native compiler toolkit that supports MLIR dialect definition and progressive lowering. The resulting quantum circuits are synthesized and validated using the Qiskit SDK (version 2.2.1) [29], running on Python 3.12.3. C programs metrics, are measured using the Lizard analyzer (version 1.17.31) [35] and Valgrind/Callgrind (version 3.25.1) [25, 26]. All experiments were executed on a workstation equipped with an Intel Core i7-12700H CPU @ 2.30 GHz and 16 GB of RAM.

4.2 Performance Analysis

To benchmark our toolchain, we follow two complementary directions. First, we evaluate the quality of the synthesized quantum circuits in terms of quantum resource usage. Second, we assess the

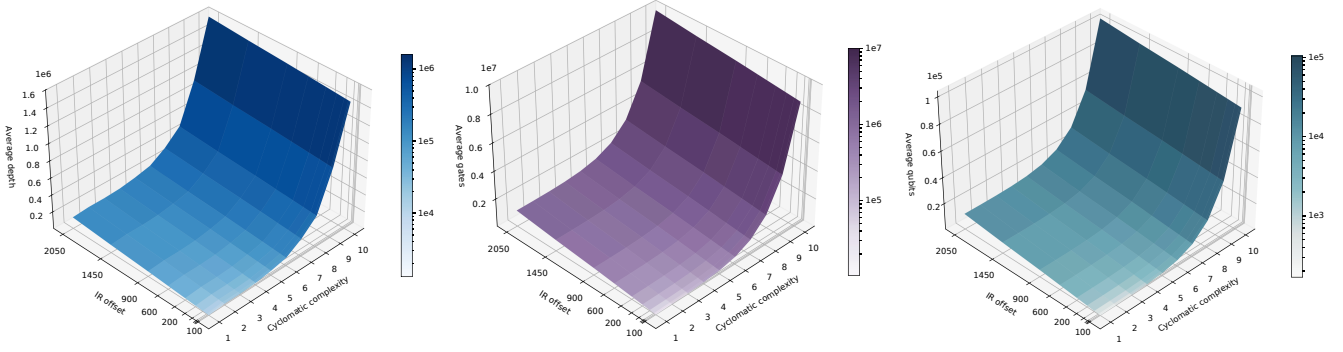


Figure 5: Heatmaps of quantum resource metrics as functions of cyclomatic complexity and instruction-count offset (Ir_off). From left to right: circuit depth, total gate count, and number of logical qubits. All metrics exhibit an approximately linear increase along the Ir_off axis and an exponential dependence along the CC axis.

compilation resource requirements of the synthesis process itself, specifically computational time and memory usage.

To characterize the input C programs, we employ two widely adopted metrics of program complexity: the instruction count (IC) and the cyclomatic complexity (CC). IC is measured on the binaries C files compiled targeting a x86-64 architecture; in our experiments IC ranges from 100 to 2000. To quantify control-flow complexity we use the CC defined as $CC = E - N + 2P$, where E is the number of edges, N the number of nodes, and P the number of connected components in the program control-flow graph [21]. CC measures the number of linearly independent paths in the control flow, thus serving as a structural indicator of program complexity. Lizard analyzer directly computes the CC value from the program control-flow graph, which for the tests ranges between 1 and 10. For each CC value a baseline program was identified. The difference between the total instruction count of a given program and the baseline defines the instruction-count offset, denoted as the Ir_off . The synthetic C programs are generated by varying the number of straight-line arithmetic instructions placed both inside and outside conditional branches. This setup ensures that, as the number of independent paths increases (i.e., higher CC), the amount of sequential operations can be adjusted independently. Consequently, these two metrics together span a comprehensive space of control-flow and instruction-level complexity.

Quantum Resource Usage. We measure all quantum metrics from the OpenQASM file of the synthesized circuit, specifically the number of virtual qubits, the number of gates, and the circuit depth. All metrics are expressed considering the circuit expressed using the universal Clifford+T quantum gate set, which is regarded as one of the most promising candidates for supporting next-generation error-correction protocols [2]. These quantities respectively represent: the circuit width, which can be seen as analogous to classical memory usage; the total number of quantum operations, roughly corresponding to the instruction count; and the circuit critical path length, which can be interpreted as the total computational time assuming a fixed execution time per quantum gate. Together, they provide a comprehensive characterization of the quantum resources required for each benchmark.

The evaluation of the quantum metrics reveals two distinct trends, shown in Fig. 5. When measured against the Ir_off , the qubit count, gate count, and circuit depth all increase linearly. This trend reflects the semantics of the compilation process, as sequential classical instructions translate proportionally into sequential quantum operations. In contrast, when measured against the CC, the same metrics exhibit exponential growth. This trend arises from the way control flow must be represented in quantum circuits. In a classical execution model, only one branch of an `if-else` statement or loop body is executed at runtime, depending on the evaluated condition. In the quantum setting, however, when mid-circuit measurements are not permitted, all possible computational paths must be emitted, with each branch realized as a controlled subcircuit. Consequently, as CC increases, the number of independent paths grows exponentially, and the quantum representation must unroll and embed all of them.

Classical Resource Usage. To measure classical resource usage, we use CPU time and peak Resident Set Size (RSS). The CPU time is defined as the sum of user and system execution times of the compiler process, while the RSS represents the maximum memory footprint reached during compilation. All measurements are performed in single-threaded mode to minimize variance and avoid interference across runs. To determine the scaling profile, each dataset is fitted against linear, quadratic, and exponential regression models. Linear and quadratic fits are computed using least-squares fitting, whereas exponential trends are estimated via nonlinear least squares. The goodness of each fit is evaluated using the coefficient of determination R^2 , defined as $R^2 = 1 - \sum_i (y_i - \hat{y}_i)^2 / \sum_i (y_i - \bar{y})^2$, where y_i are the observed values, $\hat{y}_i$ the fitted values, and $\bar{y}$ their mean. An R^2 value close to 1 indicates that the regression model explains most of the variance in the data.

We analyze both metrics from two perspectives. First, we investigate how classical compilation resources scale with the size of the synthesized quantum circuits, using circuit depth as a representative aggregate metric (Fig. 6, top). Circuit depth is chosen because all quantum resource quantities vary coherently across the (Ir_off , CC) grid (Fig. 5), making it a reliable indicator of overall quantum resource usage. Under this parameterization, CPU time

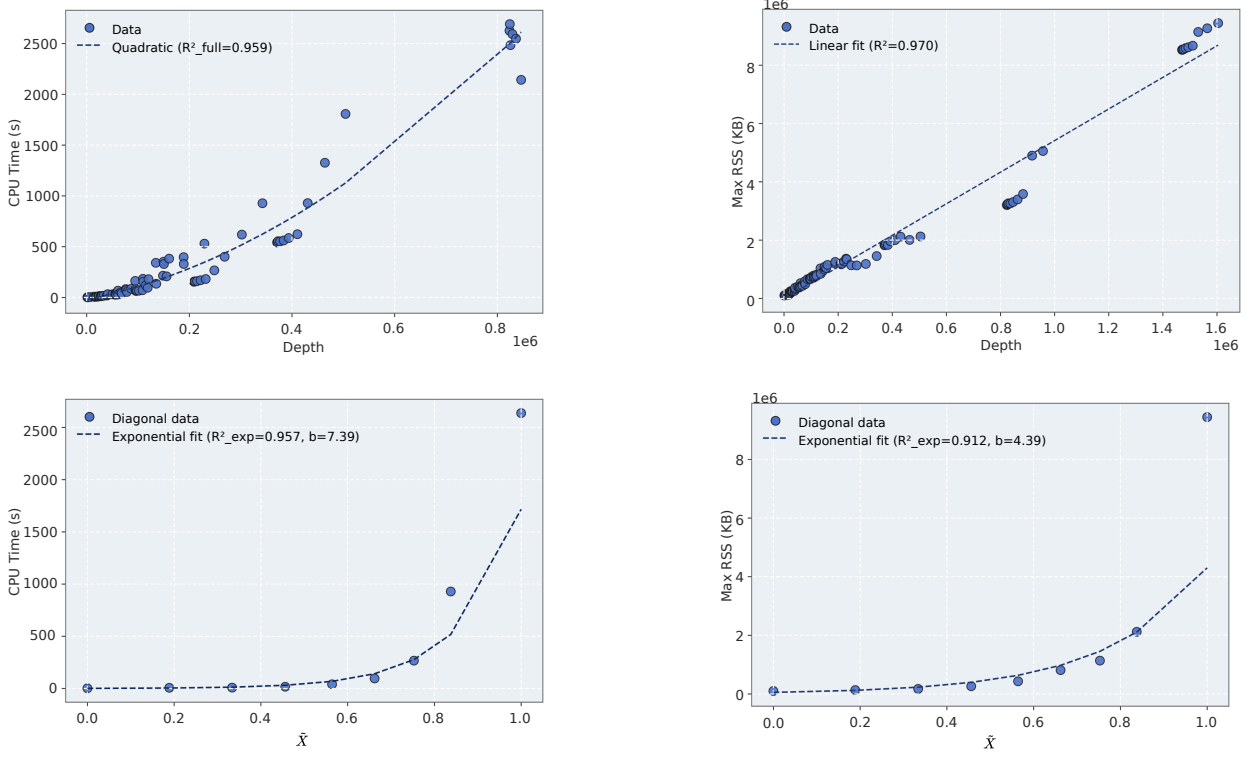


Figure 6: CPU time (left) and peak RSS (right) as functions of circuit depth (top) and of the composite complexity metric $\tilde{X}$ (bottom). CPU time exhibits quadratic growth with circuit depth and an exponential increase with complexity, whereas peak RSS grows linearly with depth and follows a more moderate exponential scaling with complexity.

scales quadratically with circuit depth ($R^2 = 0.959$), while peak RSS grows linearly ($R^2 = 0.970$). Although memory exhibits a slight curvature over the sampled range, adding a quadratic term does not significantly improve the fit; the linear model adequately captures the observed trend, as confirmed by separate regressions at fixed CC values.

Second, we analyze the combined impact of instruction-level and control-flow complexity by defining a composite metric. Specifically, we compute the product between Ir_off and CC, apply a square-root transformation, and then perform normalization: $X = \sqrt{CC \cdot Ir_off}$, $\tilde{X} = (X - X_{min}) / (X_{max} - X_{min})$. Along this axis, CPU time and peak RSS exhibit approximately exponential growth (Fig. 6, bottom). CPU time increases more steeply, reflecting the combined cost of expanding control-flow branches and synthesizing the corresponding quantum subcircuits. Memory usage grows more gradually, reflecting the accumulation of IR data structures rather than their full sequential expansion. This behavior is consistent with the depth-based analysis, where CPU time also scales faster than memory, highlighting the dominant effect of control-flow complexity on compilation costs.

Limitations and Future Directions. A primary limitation of our approach is the exponential complexity associated with translating conditional constructs, as reflected by the CC. This behavior stems from the exclusion of mid-circuit measurements, which are non-reversible operations and typically require significantly more execution time on quantum hardware than standard quantum gates, as well as back-and-forth interactions with the classical controller. Evaluating whether enabling mid-circuit measurements could reduce circuit depth—at the cost of increased classical coordination—would provide a more complete understanding of the design trade-offs. The current implementation also supports only a subset of C constructs and does not yet include functions, vector operations, or array-based computations, thereby restricting the range of programs that can be synthesized. Expanding the set of supported primitives would enhance the applicability and adoption of the framework.

Furthermore, the toolchain could be extended with additional dialects targeting different abstraction levels—such as dialects for explicit ancilla-qubit management and uncomputation or for optimizing function calls—and, thanks to MLIR modular design principles, these extensions can be incorporated without modifying the entire codebase, providing significant benefits for the quantum software engineering community.

5 Conclusions

In this work, we introduce an MLIR-based compilation framework for generating quantum circuits directly from high-level C programs. Our approach defines a Quantum-Aware IR and a fully Quantum IR, preserving high-level abstractions throughout the synthesis process. By explicitly exposing control flow, our framework provides greater flexibility than previous approaches and enables additional optimization opportunities across all intermediate stages of the compilation pipeline. We also establish a clear and well-supported correlation between quantum resource usage and classical code metrics, particularly instruction count and cyclomatic complexity. Our results show that quantum resource metrics scale linearly with instruction count and exponentially with cyclomatic complexity across all measured dimensions. Finally, to assess the feasibility of our toolchain, we analyze classical resource usage—CPU time and memory footprint—as a function of circuit size, confirming that the compilation process remains practical for the targeted range of quantum circuits.

Acknowledgments

The activity was partially funded by the *Italian Centro Nazionale di Ricerca in HPC, Big Data e Quantum computing - SPOKE 10*.

References

- [1] Andris Ambainis. 2007. Quantum walk algorithm for element distinctness. *SIAM J. Comput.*, 37, 1, 210–239. doi:10.1137/S0097539705447311.
- [2] Matthew Amy, Dmitri Maslov, Michele Mosca, and Martin Roetteler. 2013. A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 32, 6, 818–830. doi:10.1109/TCAD.2013.2244643.
- [3] Li-ming Bao. 2023. *Implementing an Intermediate Representation for Quantum Computing based on MLIR*. Bachelor's Thesis. Technische Universität München.
- [4] Philippe Coussy and Adam Morawiec. 2008. *High-Level Synthesis: from Algorithm to Digital Circuit*. (1st ed.). Springer Publishing Company, Incorporated. ISBN: 1402085877.
- [5] Aden Crimmins and Sonia Lopez Alarcon. 2024. Approximate quantum array multiplier. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Vol. 01, 58–66. doi:10.1109/QCE60285.2024.00017.
- [6] Andrew Cross et al. 2022. Openqasm 3: a broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing*, 3, 3, Article 12, (Sept. 2022), 50 pages. doi:10.1145/3505636.
- [7] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. 2004. A new quantum ripple-carry addition circuit. (2004). arXiv: quant-ph/0410184 [quant-ph].
- [8] Serena Curzel et al. 2022. End-to-end synthesis of dynamically controlled machine learning accelerators. *IEEE Trans. Computers*, 71, 12, 3074–3087. doi:10.1109/TC.2022.3211430.
- [9] Amr Elsharkawy et al. 2025. Integration of quantum accelerators with high performance computing—a review of quantum programming tools. *ACM Transactions on Quantum Computing*, 6, 3, Article 24, (July 2025), 46 pages. doi:10.1145/3743149.
- [10] Mathieu Fehr, Michel Weber, Christian Ulmann, Alexandre Lopoukhine, Martin Paul Lücke, Théo Degioanni, Christos Vasiladiotis, Michel Steuwer, and Tobias Grosser. 2025. Xdsl: sidekick compilation for ssa-based compilers. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization (CGO '25)*. Association for Computing Machinery, Las Vegas, NV, USA, 179–192. ISBN: 9798400712753. doi:10.1145/3696443.3708945.
- [11] Fabrizio Ferrandi, Vito Giovanni Castellana, Serena Curzel, Pietro Fezzardi, Michele Fiorito, Marco Lattuada, Marco Minutoli, Christian Pilato, and Antonino Tumeo. 2021. Invited: bambu: an open-source research framework for the high-level synthesis of complex applications. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, (Dec. 2021), 1327–1330. doi:10.1109/DAC18074.2021.9586110.
- [12] Craig Gidney. 2019. Windowed quantum arithmetic. *arXiv preprint arXiv:1905.07682*.
- [13] Giovanni Gozzi, Michele Fiorito, Serena Curzel, Claudio Barone, Vito Giovanni Castellana, Marco Minutoli, Antonino Tumeo, and Fabrizio Ferrandi. 2025. SPARTA: high-level synthesis of parallel multi-threaded accelerators. *ACM Trans. Reconfigurable Technol. Syst.*, 18, 1, 9:1–9:30. doi:10.1145/3677035.
- [14] Jaroslav Hrdina, Dietmar Hildenbrand, and Aleš Návrat. 2022. Quantum register algebra: the mathematical language for quantum computing. *arXiv preprint arXiv:2208.02608*.
- [15] David Ittah, Thomas Häner, Vadym Kliuchnikov, and Torsten Hoeftler. 2022. Qiro: a static single assignment-based quantum program representation for optimization. *ACM Transactions on Quantum Computing*, 3, 3, Article 14, (June 2022), 32 pages. doi:10.1145/3491247.
- [16] Murat Kurt, Ayda Kaltehei, Azmi Gençten, and Selçuk Çakmak. 2024. Scalable quantum circuit design for qft-based arithmetic. *arXiv preprint arXiv:2411.00260*.
- [17] Ryan LaRose. 2019. Overview and comparison of gate level quantum software platforms. *Quantum*, 3, 130. doi:10.22331/q-2019-03-25-130.
- [18] Chris Lattner et al. 2021. Mlir: scaling compiler infrastructure for domain specific computation. In *Proceedings of the 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO '21)*. IEEE Press, Virtual Event, Republic of Korea, 2–14. ISBN: 9781728186139. doi:10.1109/CGO51591.2021.9370308.
- [19] LLVM Project. [n. d.] Clang: a C language family frontend for LLVM. <https://clang.llvm.org/>. Accessed: 2025-08-28. ().
- [20] Chao Lu, Christian Pilato, and Kanad Basu. 2024. Qhls: an hls framework to convert high-level descriptions to quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43, 10, 3015–3026.
- [21] T.J. McCabe. 1976. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2, 4, 308–320. doi:10.1109/TSE.1976.233837.
- [22] Alexander McCaskey and Thien Nguyen. 2021. A mlir dialect for quantum assembly languages. In *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 255–264.
- [23] Alexander J. McCaskey, Dmitry I. Lyakh, Eugene F. Dumitrescu, Stephen S. Powers, and Travis S. Humble. 2020. Xacc: a system-level software infrastructure for heterogeneous quantum-classical computing. *Quantum Science and Technology*, 5, 2, 024002. doi:10.1088/2058-9565/ab6bf6.
- [24] Masoud Mohseni et al. 2024. How to build a quantum supercomputer: scaling from hundreds to millions of qubits. *CoRR*, abs/2411.10406. arXiv: 2411.10406. doi:10.48550/ARXIV.2411.10406.
- [25] Nicholas Nethercote and Julian Seward. 2007. Valgrind: a framework for heavy-weight dynamic binary instrumentation. In *Proceedings of the 2007 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, 89–100.
- [26] Nicholas Nethercote and Julian Seward. 2003. Valgrind: a program supervision framework. *Electronic Notes in Theoretical Computer Science*, 89, 2, 44–66. RV '2003, Run-time Verification (Satellite Workshop of CAV '03). doi:https://doi.org/10.1016/S1571-0661(04)81042-9.
- [27] Michael A. Nielsen and Isaac L. Chuang. 2011. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press. ISBN: 9781107002173.
- [28] Arun K. Pati and Samuel L. Braunstein. 2000. Impossibility of deleting an unknown quantum state. *Nature*, 404, 6774, 164–165. doi:10.1038/404130a0.
- [29] Qiskit Development Team. 2025. Qiskit: an open-source framework for quantum computing. <https://qiskit.org>. Accessed: 2025-07-30. (2025).
- [30] Engin Şahin. 2020. Quantum arithmetic operations based on quantum fourier transform on signed integers. *International Journal of Quantum Information*, 18, 7, 2050019. doi:10.1142/S0219749920500197.
- [31] Peter W. Shor. 1997. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM J. Comput.*, 26, 5, 1484–1509. doi:10.1137/S0097539795293172.
- [32] Robert S. Smith, Michael J. Curtis, and William J. Zeng. 2016. A practical quantum instruction set architecture. *arXiv preprint arXiv:1608.03355*.
- [33] Krysta M. Svore et al. 2018. Q#: Enabling scalable quantum computing and development with a high-level domain-specific language. Tech. rep. Microsoft. <https://www.microsoft.com/en-us/research/publication/q-enabling-scalable-quantum-computing-and-development-with-a-high-level-dsl/>.
- [34] W. K. Wootters and W. H. Zurek. 1982. A single quantum cannot be cloned. *Nature*, 299, 5886, 802–803. doi:10.1038/299802a0.
- [35] T. Yin. 2019. Lizard: an extensible cyclomatic complexity analyzer. <https://github.com/terryyin/lizard>. Accessed: YYYY-MM-DD. (2019).